Laboratorio UM

Bootstrap 5 en Visual Studio Code

Proyecto integrador: página web del Laboratorio de Robótica

Técnicas para el Desarrollo de FrontEnd — Unidad 1

4 sesiones de 80 minutos · HTML5 + CSS3 + Bootstrap 5

Ingeniería en Sistemas Computacionales

Cómo usar este tutorial

Este documento está diseñado para que el estudiante abra Visual Studio Code y trabaje directamente sobre él, sesión por sesión. El proyecto de las cuatro sesiones es único y acumulativo: la página informativa de un laboratorio universitario de robótica, que crece progresivamente desde una simple rejilla hasta un sitio completo con navegación, componentes interactivos y formulario de contacto.

Se asume que el estudiante ya conoce, de sesiones previas de la unidad: qué es Bootstrap, cómo integrarlo por CDN o de forma local, el concepto general de clases de Bootstrap y el uso de container. Estos temas no se repiten como contenido nuevo, solo se retoman como punto de partida.

Principio pedagógico general: primero comprender, después observar, después modificar, después construir y finalmente resolver un problema por cuenta propia. Cada paso del tutorial sigue esta lógica mediante las secciones Qué vamos a hacer → Código → Dónde colocarlo → Qué deberías observar → ¿Por qué funciona?

Mapa general de las 4 sesiones

Sesión	Tema principal	Producto
1	Sistema de layout (grid)	Encabezado y fila de proyectos destacados, responsive.
2	Componentes visuales	Proyectos convertidos en cards con botones, badges y un alert de bienvenida.
3	Navegación e interactividad	Navbar responsive con menú colapsable, dropdown y accordion de detalle técnico.
4	Integración y proyecto final	Sitio completo: navbar, proyectos, equipo, formulario de contacto y footer.

Entorno de desarrollo y extensiones recomendadas de VS Code

A lo largo de las cuatro sesiones se utiliza únicamente: Visual Studio Code, HTML5, CSS3 y Bootstrap 5 vía CDN. JavaScript se usa solo cuando un componente de Bootstrap lo requiere (navbar colapsable, accordion, alert cerrable). No se introducen frameworks adicionales, backend ni bases de datos.

Extensión	Para qué sirve	Por qué ayuda	Nivel
Live Server	Levanta un servidor local con recarga automática al guardar.	Permite ver cada cambio de inmediato sin refrescar manualmente.	Indispensable
Prettier	Formatea automáticamente el código HTML/CSS.	Mantiene indentación consistente, más fácil de leer y depurar.	Recomendable
Auto Rename Tag	Renombra automáticamente la etiqueta de cierre al editar la de apertura.	Evita errores de etiquetas HTML mal cerradas, muy comunes al inicio.	Recomendable

Extensión	Para qué sirve	Por qué ayuda	Nivel
IntelliSense for CSS class names	Autocompleta nombres de clases de Bootstrap mientras se escribe.	Reduce errores de escritura en clases como col-md-4 o btn-primary.	Recomendable
Bootstrap 5 Quick Snippets	Inserta fragmentos predefinidos de componentes de Bootstrap.	Acelera la escritura de estructuras repetitivas (cards, navbar).	Opcional

Buenas prácticas transversales

- HTML semántico: usar nav, footer, section cuando corresponda, en lugar de solo div.
- Indentación consistente y nombres de clase claros.
- Comprender el código en lugar de copiarlo sin analizarlo.
- Accesibilidad básica integrada en cada ejercicio: atributos alt, aria-label, label asociado a inputs, uso correcto de encabezados.
- Responsive design verificado activamente con las herramientas de desarrollador del navegador en cada sesión, no solo al final.

Sesión 1 — El sistema de rejilla: la columna vertebral de Bootstrap

1. Título

Construyendo el esqueleto responsive del laboratorio de robótica

2. Objetivo de aprendizaje

Al finalizar la sesión, el estudiante será capaz de:

- Identificar los elementos del sistema de rejilla de Bootstrap 5 (container, row, col).
- Aplicar clases responsivas de columnas (col-sm, col-md, col-lg, col-xl) para distribuir contenido.
- Construir la primera sección estructural de la página del laboratorio de robótica utilizando el grid.
- Modificar el comportamiento responsive de una fila probando distintos anchos de pantalla.

3. Conocimientos previos

Esta sesión retoma, sin volver a explicarlos como contenido principal, los temas ya trabajados: qué es Bootstrap, integración vía CDN, integración local, el concepto general de clases de Bootstrap y el uso de container. Estos conceptos se usarán como punto de partida inmediato.

4. Conceptos nuevos

- Grid system: sistema de 12 columnas que organiza el contenido en filas y columnas.
- row: contenedor flexible que agrupa columnas y aplica márgenes negativos para compensar el gutter.
- col / col-*: unidades de columna que reparten el ancho disponible.
- Breakpoints (sm, md, lg, xl, xxl): puntos de quiebre donde el layout cambia según el ancho de pantalla.
- Columnas responsivas (col-md-4, col-lg-3, etc.): permiten que una columna ocupe distinto número de unidades según el dispositivo.
- Gutters: espacios de separación horizontal/vertical entre columnas (g-*, gx-*, gy-*).
- Mobile first: las clases sin infijo (col-4) aplican desde el móvil hacia arriba, y se sobreescriben con breakpoints mayores.

5. Distribución de los 80 minutos

Tiempo	Actividad	Descripción
5–10 min	Recuperación de conocimientos previos	Repaso rápido oral: ¿qué es container?, ¿por qué usamos un CDN? Se muestra el archivo base ya creado en clases anteriores.
10–15 min	Explicación de conceptos nuevos	Explicación del grid de 12 columnas, row, col y breakpoints usando el método 'Clase de Bootstrap' (ver Metodología).
20–25 min	Demostración guiada	El profesor construye en vivo el encabezado y la primera fila de tarjetas de proyectos del

Tiempo	Actividad	Descripción
		laboratorio, mostrando ANTES/DESPUÉS.
25–30 min	Práctica del estudiante	Los estudiantes replican y amplían la fila con sus propias columnas, probando distintos col-md/col-lg.
5–10 min	Reto de consolidación	Construir una fila de 3 columnas que se convierta en 1 columna en móvil y 3 en escritorio, sin ayuda directa.
5 min	Cierre y comprobación	Verificación visual proyectada de 2-3 pantallas de estudiantes; se listan dudas para la siguiente sesión.

6. Preparación del entorno

Antes de comenzar, cada estudiante debe preparar su entorno de la siguiente manera:

1. Crear una carpeta llamada laboratorio-robotica en el escritorio o en su carpeta de proyectos.
2. Dentro de esa carpeta, crear un archivo index.html.
3. Crear una subcarpeta css y dentro un archivo styles.css (vacío por ahora, se usará en la Sesión 2).
4. Abrir la carpeta completa laboratorio-robotica en Visual Studio Code (Archivo → Abrir carpeta…), nunca abrir solo el archivo suelto.
5. Instalar y activar la extensión Live Server si aún no se ha hecho.
6. Verificar que index.html ya tiene la plantilla base de Bootstrap 5 vía CDN (trabajada en sesiones previas) y un container vacío en el <body>.
7. Ejecutar el proyecto con clic derecho sobre index.html → 'Open with Live Server' y confirmar que el navegador se abre correctamente.

7. Tutorial paso a paso

Paso 1 — Definir el objetivo del proyecto

Qué vamos a hacer

Vamos a construir, a lo largo de las 4 sesiones, la página web informativa de un laboratorio universitario de robótica, con secciones para presentar proyectos, equipo y formulario de contacto. Hoy nos enfocamos únicamente en la estructura de layout.

Dónde colocarlo

No requiere código todavía; es una discusión breve con el grupo sobre las secciones que tendrá el sitio (encabezado, proyectos destacados, equipo, contacto, pie de página).

Paso 2 — Confirmar el container base

Qué vamos a hacer

Recordamos brevemente cómo container centra y limita el ancho del contenido, y lo dejamos como punto de partida.

Código — index.html (estructura ya existente)

```
<body>
  <div class="container">

  </div>
</body>
```

Qué deberías observar

Una página en blanco con márgenes laterales; el contenido nunca toca el borde de la ventana.

¿Por qué funciona?

container aplica un ancho máximo distinto en cada breakpoint y centra el bloque con márgenes automáticos, preparando el terreno para el grid.

Paso 3 — Agregar la primera row

Qué vamos a hacer

Insertamos una fila dentro del container para alojar el encabezado del laboratorio.

ANTES

```
<div class="container">

</div>
```

DESPUÉS

```
<div class="container">
  <div class="row">

  </div>
</div>
```

Qué deberías observar

Visualmente no cambia nada todavía (row sin columnas no se nota), pero al inspeccionar con las herramientas de desarrollador aparecen márgenes negativos en el elemento .row.

¿Por qué funciona?

row es un contenedor flex que prepara el terreno para las columnas y compensa, mediante márgenes negativos, el padding interno que tendrán las columnas (gutter).

Paso 4 — Agregar columnas dentro de la row

Qué vamos a hacer

Dividimos la fila en dos columnas: una para el título del laboratorio y otra para una breve descripción.

ANTES

```
<div class="row">

</div>
```

DESPUÉS

```
<div class="row">
  <div class="col-md-6">
    <h1>Laboratorio de Robótica</h1>
  </div>
  <div class="col-md-6">
    <p>Diseño, construcción y programación de sistemas robóticos aplicados.</p>
  </div>
</div>
```

Qué deberías observar

En pantallas grandes, el título y la descripción se ven lado a lado, cada uno ocupando la mitad del ancho. En pantallas pequeñas, se apilan uno debajo del otro.

¿Por qué funciona?

col-md-6 significa: 'ocupa 6 de las 12 columnas a partir del breakpoint md (≥768px)'. Como no se especifica una clase base (col-12), Bootstrad asume col-12 por defecto en móvil según el enfoque mobile first — en este caso, sin clase base explícita, cada columna se comporta como col-12 en pantallas muy pequeñas.

Paso 5 — Construir la fila de proyectos destacados

Qué vamos a hacer

Agregamos una segunda row con tres columnas para mostrar tres proyectos del laboratorio (por ejemplo: robot móvil diferencial, CanSAT y brazo robótico).

Código — nueva sección debajo del encabezado

```
<div class="row mt-5">
  <div class="col-sm-6 col-lg-4">
    <h3>Robot móvil diferencial</h3>
    <p>Plataforma con dos motores independientes para navegación autónoma.</p>
  </div>
  <div class="col-sm-6 col-lg-4">
    <h3>Proyecto CanSAT</h3>
    <p>Simulación de un satélite en formato lata para telemetría atmosférica.</p>
  </div>
  <div class="col-sm-6 col-lg-4">
    <h3>Brazo robótico</h3>
    <p>Manipulador de varios grados de libertad para tareas de precisión.</p>
  </div>
</div>
```

Dónde colocarlo

Dentro del mismo container, justo después de la row del encabezado creada en el Paso 4.

Qué deberías observar

En escritorio grande (lg) se ven 3 columnas por fila; en tablet (sm) se ven 2 columnas y la tercera pasa a la siguiente línea; en móvil, cada tarjeta ocupa todo el ancho.

¿Por qué funciona?

Al combinar col-sm-6 col-lg-4 se definen dos comportamientos distintos según el breakpoint: 6/12 (dos columnas) desde sm, y 4/12 (tres columnas) desde lg. Bootstrap aplica el breakpoint más específico que coincida con el ancho actual.

Paso 6 — Ajustar el espacio entre columnas (gutters)

Qué vamos a hacer

Aumentamos el espacio horizontal entre las tarjetas de proyectos para que se vean menos apretadas.

ANTES

```
<div class="row mt-5">
```

DESPUÉS

```
<div class="row mt-5 gx-4">
```

Qué deberías observar

El espacio horizontal entre columnas aumenta notablemente.

¿Por qué funciona?

Las utilidades gx-* controlan el gutter horizontal mediante variables CSS (--bs-gutter-x), sin necesidad de escribir CSS personalizado.

8. Experimento rápido

Cambia col-lg-4 por col-lg-6 en las tres tarjetas de proyectos.

- ¿Qué cambió en el número de columnas por fila en pantallas grandes?
- ¿Por qué la suma de 6+6+6 = 18 hace que la tercera tarjeta 'salte' a una nueva línea?
- Reduce el ancho de la ventana del navegador poco a poco: ¿en qué punto exacto cambia el layout?

9. Reto guiado

Problema

Construir una fila de pie de página con tres columnas: logotipo/nombre del laboratorio, enlaces rápidos y datos de contacto.

Requisitos

- En móvil, las tres columnas deben apilarse en una sola columna cada una.
- En tablet (md), deben mostrarse dos columnas por fila.
- En escritorio (lg), las tres deben mostrarse en la misma fila.

Pistas

- Piensa qué clase base representa 'ocupar todo el ancho' en móvil.
- Recuerda que puedes combinar varias clases de columna en el mismo div.

Solución propuesta

Código

```
<div class="row mt-5">
  <div class="col-12 col-md-6 col-lg-4">
    <h5>Laboratorio de Robótica</h5>
  </div>
  <div class="col-12 col-md-6 col-lg-4">
```

```
    <h5>Enlaces</h5>
  </div>
  <div class="col-12 col-md-6 col-lg-4">
    <h5>Contacto</h5>
  </div>
</div>
```

La clase base col-12 asegura el comportamiento en móvil (una columna por fila); col-md-6 introduce dos columnas por fila desde tablet; col-lg-4 introduce tres columnas por fila desde escritorio.

10. Errores frecuentes

- Colocar col-* directamente dentro de container, sin una row intermedia: rompe el sistema de márgenes/gutters.
- Anidar columnas sin envolverlas en una nueva row: una columna dentro de otra columna necesita su propio row interno.
- Escribir mal el nombre de la clase (ej. col-md6 sin guion): Bootstrap ignora silenciosamente la clase y la columna no se comporta como se espera.
- Olvidar la clase base (col-12) y suponer que col-lg-4 funciona igual en todos los tamaños de pantalla.
- No guardar el archivo antes de revisar en el navegador (Live Server no refresca contenido no guardado).
- Confundir gx- (gutter horizontal) con gy- (gutter vertical) al espaciar tarjetas que se apilan.

11. Mini evaluación

1. ¿Cuántas columnas conforman el sistema de rejilla de Bootstrap 5?
2. ¿Qué elemento HTML debe envolver siempre a las columnas antes de colocarlas dentro de un container?
3. Si una fila contiene col-md-4 tres veces, ¿cómo se distribuyen en una pantalla menor a 768px?
4. Dado el código col-sm-6 col-xl-3, predice cuántas columnas por fila se verán en una pantalla de 1400px de ancho.
5. Un estudiante escribió ... directamente dentro de <body>, sin container. ¿Qué problema visual podría presentarse?
6. ¿Qué utilidad de Bootstrap se usa para aumentar el espacio horizontal entre columnas?
7. Explica con tus palabras qué significa 'mobile first' en el contexto del grid.

12. Evidencia de aprendizaje

- Captura de pantalla del sitio en tres anchos distintos (móvil, tablet, escritorio) usando las herramientas de desarrollador.
- Archivo index.html con el encabezado, la fila de proyectos y el pie de página construidos durante la sesión.
- Solución escrita (código) del reto guiado del pie de página.

13. Cierre

Lo que aprendimos

- El sistema de rejilla de 12 columnas y su lógica mobile first.

- Uso de container, row y col-* con breakpoints.
- Control de espaciado entre columnas mediante utilidades gx-/gy-.

Lo que utilizaremos en la siguiente sesión

La estructura de filas y columnas construida hoy será el 'esqueleto' sobre el cual, en la Sesión 2, se colocarán componentes visuales de Bootstrap: tarjetas (cards), botones, alerts y utilidades de espaciado y color para cada proyecto del laboratorio.

Nota para el profesor

- Confusión frecuente: alumnos que suman más de 12 en una fila y no entienden por qué 'se rompe' el layout; aprovechar para reforzar la aritmética del grid.
- Observar mientras trabajan: quiénes olvidan la row intermedia al anidar columnas.
- Errores que delatan falta de comprensión: usar col-md-4 sin nunca haber probado a reducir la ventana del navegador (no verificaron el comportamiento responsive).
- Preguntas de verificación: '¿Qué pasaría si cambio col-lg-4 por col-lg-3 en las tres tarjetas?'
- Contenido omitible si el grupo avanza lento: el ajuste fino de gx-/gy- puede posponerse a la Sesión 2.
- Actividad adicional si terminan antes: pedirles agregar una cuarta tarjeta de proyecto y observar cómo se acomoda automáticamente en la rejilla.

Sesión 2 — Componentes visuales: dando identidad al laboratorio

1. Título

Tarjetas, botones y utilidades: vistiendo el proyecto

2. Objetivo de aprendizaje

- Identificar la diferencia entre componentes, utilities y clases estructurales de Bootstrap.
- Aplicar cards, buttons, alerts y badges para presentar la información del laboratorio.
- Utilizar utilidades de espaciado, color, bordes y sombras para refinar el diseño.
- Integrar los nuevos componentes dentro de la rejilla construida en la Sesión 1.

3. Conocimientos previos

Se reutiliza directamente la estructura de container, row y col-* construida en la Sesión 1. Los proyectos destacados (robot móvil diferencial, CanSAT, brazo robótico) ahora se convertirán en tarjetas visuales dentro de las columnas ya existentes.

4. Conceptos nuevos

- Cards: componente contenedor con encabezado, cuerpo, imagen y pie, ideal para presentar unidades de contenido.
- Buttons: clases btn + variante de color (btn-primary, btn-outline-secondary, etc.).
- Alerts: mensajes contextuales de color (éxito, advertencia, información, peligro).
- Badges: pequeñas etiquetas para estados o categorías (ej. 'Activo', 'En desarrollo').
- Typography utilities: clases para tamaño, peso y alineación de texto.
- Spacing utilities (m-*, p-*): márgenes y rellenos estandarizados en una escala de 0 a 5.
- Colors, borders, shadows: utilidades bg-*, text-*, border-*, shadow-* para reforzar jerarquía visual.

5. Distribución de los 80 minutos

Tiempo	Actividad	Descripción
5–10 min	Recuperación de conocimientos previos	Se abre el proyecto de la Sesión 1 y se revisa en conjunto la rejilla de proyectos ya construida.
10–15 min	Explicación de conceptos nuevos	Diferencia entre componente (card), utility (p-3, shadow-sm) y clase estructural (row, col); ejemplos mínimos de cada uno.
20–25 min	Demostración guiada	El profesor convierte, en vivo, la primera tarjeta de proyecto (robot móvil diferencial) en una card completa con botón y badge.
25–30 min	Práctica del estudiante	Los estudiantes convierten las dos tarjetas restantes (CanSAT, brazo

Tiempo	Actividad	Descripción
		robótico) siguiendo el mismo patrón.
5–10 min	Reto de consolidación	Agregar un alert de bienvenida en la parte superior de la página, cerrable por el usuario.
5 min	Cierre y comprobación	Revisión visual proyectada; se anota qué componentes generaron más dudas.

6. Preparación del entorno

1. Abrir la carpeta laboratorio-robotica (la misma de la Sesión 1) en Visual Studio Code.
2. Verificar que index.html conserva el encabezado y la fila de tres proyectos de la sesión anterior.
3. No es necesario crear archivos nuevos; todo el trabajo de hoy ocurre dentro de index.html.
4. Confirmar que Live Server sigue activo o volver a iniciarlo sobre index.html.

7. Tutorial paso a paso

Paso 1 — Diferenciar componente, utility y clase estructural

Qué vamos a hacer

Antes de escribir código, aclaramos con ejemplos cortos: row/col son clases estructurales (organizan el espacio); card es un componente (una pieza de UI completa con su propio HTML interno); p-3 o shadow-sm son utilities (modifican una sola propiedad visual, se combinan libremente).

¿Por qué funciona?

Comprender esta diferencia evita que el estudiante memorice clases sueltas sin entender que puede combinarlas: una card puede llevar utilities de sombra, color o espaciado sin dejar de ser una card.

Paso 2 — Convertir la primera tarjeta de proyecto en una card

Qué vamos a hacer

Reemplazamos el contenido simple de la primera columna de proyectos por una card completa.

ANTES

```
<div class="col-sm-6 col-lg-4">
  <h3>Robot móvil diferencial</h3>
  <p>Plataforma con dos motores independientes para navegación autónoma.</p>
</div>
```

DESPUÉS

```
<div class="col-sm-6 col-lg-4">
  <div class="card shadow-sm h-100">
    <div class="card-body">
      <h3 class="card-title">Robot móvil diferencial</h3>
      <span class="badge bg-success mb-2">Activo</span>
      <p class="card-text">Plataforma con dos motores independientes para navegación autónoma.</p>
      <a href="#" class="btn btn-primary">Ver detalles</a>
```

```
    </div>
  </div>
</div>
```

Qué deberías observar

La información ahora aparece dentro de un recuadro con sombra suave, con una etiqueta verde 'Activo' y un botón morado 'Ver detalles'.

¿Por qué funciona?

card define el contenedor visual; card-body agrega relleno interno estándar; shadow-sm y h-100 son utilities que añaden profundidad y hacen que todas las tarjetas de la fila midan lo mismo aunque su texto sea de distinta longitud.

Paso 3 — Replicar el patrón en las otras dos tarjetas

Qué vamos a hacer

Aplicamos el mismo patrón de card a los proyectos CanSAT y Brazo robótico, cambiando el color del badge según el estado del proyecto.

Código — CanSAT

```
<div class="col-sm-6 col-lg-4">
  <div class="card shadow-sm h-100">
    <div class="card-body">
      <h3 class="card-title">Proyecto CanSAT</h3>
      <span class="badge bg-warning text-dark mb-2">En pruebas</span>
      <p class="card-text">Simulación de un satélite en formato lata para telemetría
atmosférica.</p>
      <a href="#" class="btn btn-outline-primary">Ver detalles</a>
    </div>
  </div>
</div>
```

Dónde colocarlo

En la segunda y tercera columna de la fila de proyectos, reemplazando el contenido simple igual que en el Paso 2.

Qué deberías observar

Tres tarjetas alineadas y de igual altura, cada una con un badge de color distinto según su estado.

¿Por qué funciona?

bg-warning text-dark combina dos utilities de color para garantizar buen contraste; btn-outline-primary es una variante de botón con borde en vez de relleno, útil para acciones secundarias.

Paso 4 — Agregar un alert de bienvenida

Qué vamos a hacer

Colocamos un mensaje contextual antes del encabezado para dar la bienvenida a los visitantes del sitio.

Código — dentro del container, antes de la primera row

```
<div class="alert alert-info alert-dismissible fade show" role="alert">
  Bienvenido al sitio oficial del Laboratorio de Robótica.
```

```
  <button type="button" class="btn-close" data-bs-dismiss="alert" aria-label="Cerrar"></button>
</div>
```

Qué deberías observar

Una franja azul claro con el mensaje y una 'X' que, al hacer clic, oculta el aviso con una animación suave.

¿Por qué funciona?

alert-dismissible junto con el botón data-bs-dismiss="alert" activan comportamiento interactivo de Bootstrap; esto requiere que el JavaScript de Bootstrap esté cargado (se confirma en el Paso 5).

Paso 5 — Confirmar que el JavaScript de Bootstrap está disponible

Qué vamos a hacer

Verificamos que el <script> del bundle de Bootstrap (con Popper incluido) esté presente antes de cerrar </body>, ya que el botón de cerrar del alert lo requiere.

Código — antes de </body>

```
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
```

¿Por qué funciona?

El CSS de Bootstrap solo define apariencia; el comportamiento interactivo (cerrar un alert, abrir un dropdown, etc.) depende de su JavaScript. Sin este script, el botón 'X' sería visible pero no funcional.

8. Experimento rápido

Cambia badge bg-success por badge bg-danger en la primera tarjeta.

- ¿Qué transmite visualmente el cambio de color al usuario?
- ¿Por qué es importante no usar el color al azar, sino con un significado consistente (verde=activo, rojo=detenido)?

9. Reto guiado

Problema

Agregar una cuarta card fuera de la fila de proyectos: una tarjeta de 'Únete al laboratorio' con un botón de llamada a la acción.

Requisitos

- Debe usar la clase card con al menos un card-body.
- Debe incluir un botón con una variante de color distinta a las ya usadas (por ejemplo btn-success).
- Debe llevar una utility de espaciado (mt-4 o similar) para separarla del resto del contenido.

Pistas

- Puedes colocarla dentro de su propia row y col-12 para que ocupe todo el ancho.
- Revisa la documentación de variantes de botones si dudas del nombre exacto de la clase.

Solución propuesta

Código

```
<div class="row mt-4">
  <div class="col-12">
    <div class="card shadow-sm bg-light">
      <div class="card-body text-center">
        <h4>¿Quieres unirte al laboratorio?</h4>
        <p>Buscamos estudiantes interesados en robótica de servicio y sistemas embebidos.</p>
        <a href="#contacto" class="btn btn-success">Contáctanos</a>
      </div>
    </div>
  </div>
</div>
```

10. Errores frecuentes

- Olvidar card-body: el contenido queda pegado al borde de la tarjeta, sin relleno.
- Usar btn sin una variante de color (btn-primary, btn-success, etc.): el botón se ve sin estilo definido.
- Colocar el alert-dismissible sin el atributo data-bs-dismiss="alert" en el botón: la 'X' aparece pero no cierra nada.
- Cargar el bundle de JavaScript de Bootstrap antes que el contenido del <body>: puede provocar que el script no encuentre los elementos aún.
- Confundir bg-warning (fondo) con text-warning (texto) y obtener un resultado visual distinto al esperado.
- No usar h-100 en tarjetas dentro de una misma row: quedan de alturas distintas si el texto varía en longitud.

11. Mini evaluación

1. Explica, con tus propias palabras, la diferencia entre un componente y una utility en Bootstrap.
2. ¿Qué clase agrega el relleno interno estándar dentro de una card?
3. ¿Qué combinación de clases hace que un botón se vea con solo el borde de color, sin relleno?
4. Un estudiante ve que el botón de cerrar de su alert no funciona. Menciona dos posibles causas.
5. ¿Qué utility usarías para dar sombra suave a una tarjeta?
6. Dado Detenido, ¿qué información transmite al usuario y por qué el color es relevante?

12. Evidencia de aprendizaje

- Archivo index.html con las tres cards de proyectos, el alert de bienvenida y la card del reto guiado.
- Captura de pantalla mostrando el alert antes y después de cerrarlo.

13. Cierre

Lo que aprendimos

- Diferencia entre componentes, utilities y clases estructurales.
- Uso de cards, buttons, alerts y badges.
- Utilidades de espaciado, color, bordes y sombras aplicadas sobre la rejilla existente.

Lo que utilizaremos en la siguiente sesión

La estructura visual de hoy (tarjetas, botones) se integrará dentro de un sistema de navegación: en la Sesión 3 construiremos una navbar responsive que permita moverse entre las secciones del sitio, además de un componente interactivo adicional (accordion o modal) para mostrar más detalle de cada proyecto.

Nota para el profesor

- Confusión frecuente: alumnos que piensan que card-body es opcional; mostrar en vivo el resultado sin él.
- Observar mientras trabajan: quiénes copian clases de botón sin variante de color.
- Errores que indican falta de comprensión: usar utilities de color (bg-success) esperando que cambien también el borde o el texto automáticamente.
- Preguntas de verificación: '¿Qué pasa si quito shadow-sm? ¿Y si quito h-100?'
- Contenido omitible si el grupo avanza lento: el detalle de alert-dismissible puede simplificarse a un alert estático sin botón de cerrar.
- Actividad adicional si terminan antes: pedir que agreguen una imagen a una de las cards usando card-img-top.

Sesión 3 — Navegación y componentes interactivos

1. Título

Moviéndose por el sitio: navbar y contenido interactivo

2. Objetivo de aprendizaje

- Identificar los componentes de una navbar responsive de Bootstrap.
- Aplicar collapse para el comportamiento de menú hamburguesa en móvil.
- Implementar un accordion para mostrar el detalle de cada proyecto del laboratorio.
- Explicar la diferencia entre Bootstrap CSS y Bootstrap JavaScript en componentes interactivos.

3. Conocimientos previos

Se reutiliza el archivo index.html con el encabezado, el alert, la fila de proyectos con cards y la card de llamada a la acción construidos en la Sesión 2. Hoy se agrega, por encima de todo ese contenido, un sistema de navegación.

4. Conceptos nuevos

- Navbar: barra de navegación horizontal, colapsable en pantallas pequeñas.
- navbar-expand-*: define a partir de qué breakpoint la navbar se muestra expandida (no colapsada).
- navbar-toggler: botón tipo 'hamburguesa' que aparece cuando la navbar está colapsada.
- Collapse: comportamiento que oculta/muestra contenido mediante clases y atributos data-bs-*.
- Dropdown: menú desplegable dentro de la navegación.
- Accordion: panel de secciones expandibles, útil para mostrar/ocultar detalle sin recargar la página.
- Diferencia entre Bootstrap CSS (apariencia) y Bootstrap JavaScript (comportamiento: abrir, cerrar, animar).

5. Distribución de los 80 minutos

Tiempo	Actividad	Descripción
5–10 min	Recuperación de conocimientos previos	Se revisa el sitio construido hasta la Sesión 2 y se identifica dónde falta un menú de navegación.
10–15 min	Explicación de conceptos nuevos	Explicación de navbar, collapse y la diferencia CSS/JavaScript con un ejemplo mínimo proyectado.
20–25 min	Demostración guiada	El profesor construye en vivo la navbar responsive completa con enlaces a las secciones del sitio.
25–30 min	Práctica del estudiante	Los estudiantes agregan anclas (id) a sus secciones y construyen un accordion con el detalle de cada proyecto.

Tiempo	Actividad	Descripción
5–10 min	Reto de consolidación	Agregar un dropdown dentro de la navbar con enlaces a subsecciones (ej. 'Proyectos' → cada proyecto).
5 min	Cierre y comprobación	Prueba en vivo del menú hamburguesa reduciendo el ancho del navegador.

6. Preparación del entorno

1. Abrir nuevamente la carpeta laboratorio-robotica en Visual Studio Code.
2. Confirmar que el archivo index.html conserva todo el trabajo de la Sesión 2.
3. No se crean archivos nuevos; el trabajo de hoy continúa en index.html, agregando contenido antes del encabezado existente.
4. Mantener Live Server activo durante toda la sesión para ver los cambios de inmediato.

7. Tutorial paso a paso

Paso 1 — Crear la estructura base de la navbar

Qué vamos a hacer

Insertamos la navbar como primer elemento dentro de <body>, antes del container principal.

Código — justo después de <body>

```
<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
  <div class="container">
    <a class="navbar-brand" href="#">Lab. Robótica</a>
  </div>
</nav>
```

Qué deberías observar

Una barra oscura en la parte superior del sitio con el nombre del laboratorio.

¿Por qué funciona?

navbar-expand-lg indica que la navegación se mostrará expandida (horizontal) a partir del breakpoint lg; por debajo de ese ancho, se colapsará automáticamente. bg-dark y navbar-dark ajustan fondo y color de texto para buen contraste.

Paso 2 — Agregar el botón hamburguesa y el contenedor colapsable

Qué vamos a hacer

Agregamos el botón que aparecerá en pantallas pequeñas y el contenedor que se ocultará/mostrará.

ANTES

```
<div class="container">
  <a class="navbar-brand" href="#">Lab. Robótica</a>
</div>
```

DESPUÉS

```
<div class="container">
  <a class="navbar-brand" href="#">Lab. Robótica</a>
  <button class="navbar-toggler" type="button" data-bs-toggle="collapse"
          data-bs-target="#menuPrincipal" aria-controls="menuPrincipal"
          aria-expanded="false" aria-label="Abrir menú">
    <span class="navbar-toggler-icon"></span>
  </button>
  <div class="collapse navbar-collapse" id="menuPrincipal">

  </div>
</div>
```

Qué deberías observar

En pantallas anchas no se nota cambio visible aún (el contenedor colapsable está vacío). Al reducir el ancho de la ventana por debajo de lg, aparece un ícono de tres líneas horizontales.

¿Por qué funciona?

data-bs-toggle="collapse" y data-bs-target="#menuPrincipal" conectan el botón con el contenedor mediante su id; al hacer clic, el JavaScript de Bootstrap alterna la clase show que despliega el contenido.

Paso 3 — Agregar los enlaces de navegación

Qué vamos a hacer

Colocamos la lista de enlaces dentro del contenedor colapsable, apuntando a las secciones del sitio mediante anclas.

Código — dentro de #menuPrincipal

```
<ul class="navbar-nav ms-auto">
  <li class="nav-item">
    <a class="nav-link" href="#proyectos">Proyectos</a>
  </li>
  <li class="nav-item">
    <a class="nav-link" href="#equipo">Equipo</a>
  </li>
  <li class="nav-item">
    <a class="nav-link" href="#contacto">Contacto</a>
  </li>
</ul>
```

Dónde colocarlo

Agregar id="proyectos" a la row de las tarjetas de proyectos (Sesión 2) para que el enlace 'Proyectos' funcione correctamente; las secciones 'Equipo' y 'Contacto' se completarán en la Sesión 4.

Qué deberías observar

En escritorio, los tres enlaces aparecen alineados a la derecha de la navbar; en móvil, aparecen apilados dentro del menú desplegable al pulsar el botón hamburguesa.

¿Por qué funciona?

ms-auto empuja el grupo de enlaces hacia la derecha mediante márgenes automáticos flexbox; navbar-nav y nav-item/nav-link son las clases estructurales específicas de listas de navegación.

Paso 4 — Construir un accordion con el detalle de cada proyecto

Qué vamos a hacer

Debajo de la fila de cards de proyectos, agregamos un accordion que permite expandir información técnica adicional de cada proyecto sin saturar la vista inicial.

Código — nueva sección después de las cards

```
<div class="accordion mt-4" id="detalleProyectos">
  <div class="accordion-item">
    <h2 class="accordion-header">
      <button class="accordion-button collapsed" type="button"
              data-bs-toggle="collapse" data-bs-target="#colapso1">
        Robot móvil diferencial — Detalle técnico
      </button>
    </h2>
    <div id="colapso1" class="accordion-collapse collapse"
         data-bs-parent="#detalleProyectos">
      <div class="accordion-body">
        Plataforma de tracción diferencial con dos motores DC controlados
        de forma independiente, sensores de distancia y control mediante
        microcontrolador ESP32.
      </div>
    </div>
  </div>
</div>
```

Qué deberías observar

Un panel colapsado que, al hacer clic en el título, se despliega mostrando el texto de detalle con una animación de deslizamiento.

¿Por qué funciona?

data-bs-parent="#detalleProyectos" indica que este panel pertenece a un grupo de acordeón: al abrir uno, Bootstrap cierra automáticamente los demás paneles del mismo grupo.

Paso 5 — Repetir el patrón del accordion para los otros dos proyectos

Qué vamos a hacer

Agregamos dos accordion-item adicionales, cada uno con un id único (colapso2, colapso3) para CanSAT y el brazo robótico.

Código — segundo panel

```
<div class="accordion-item">
  <h2 class="accordion-header">
    <button class="accordion-button collapsed" type="button"
            data-bs-toggle="collapse" data-bs-target="#colapso2">
      Proyecto CanSAT — Detalle técnico
    </button>
  </h2>
  <div id="colapso2" class="accordion-collapse collapse"
       data-bs-parent="#detalleProyectos">
    <div class="accordion-body">
      Estructura tipo lata con sensores de presión, temperatura y GPS,
      transmitiendo telemetría a una estación terrena en tiempo real.
    </div>
  </div>
</div>
```

Qué deberías observar

Al hacer clic en el segundo panel, este se abre y el primero se cierra automáticamente.

¿Por qué funciona?

Cada panel tiene un id distinto (colapso1, colapso2) referenciado únicamente por su propio botón, pero comparten el mismo data-bs-parent, lo que agrupa su comportamiento de apertura exclusiva.

8. Experimento rápido

Quita el atributo data-bs-parent="#detalleProyectos" de los tres paneles del accordion.

- ¿Qué cambia en el comportamiento al abrir varios paneles a la vez?
- ¿Por qué ese atributo es el responsable de que solo un panel esté abierto a la vez?

9. Reto guiado

Problema

Agregar un dropdown dentro de la navbar llamado 'Más' con dos opciones: 'Documentación' y 'Redes sociales'.

Requisitos

- Debe integrarse como un nav-item más dentro de la lista navbar-nav ya existente.
- Debe usar dropdown-toggle y un dropdown-menu con al menos dos dropdown-item.
- Debe funcionar correctamente tanto en escritorio como dentro del menú colapsado en móvil.

Pistas

- El dropdown también usa data-bs-toggle, pero con el valor "dropdown" en vez de "collapse".
- Revisa que el <li> que contiene el dropdown tenga la clase dropdown.

Solución propuesta

Código

```
<li class="nav-item dropdown">
  <a class="nav-link dropdown-toggle" href="#" role="button"
     data-bs-toggle="dropdown" aria-expanded="false">
    Más
  </a>
  <ul class="dropdown-menu">
    <li><a class="dropdown-item" href="#">Documentación</a></li>
    <li><a class="dropdown-item" href="#">Redes sociales</a></li>
  </ul>
</li>
```

10. Errores frecuentes

- El id del botón (data-bs-target) no coincide exactamente con el id del contenedor colapsable: el menú o el panel no responde al clic.
- Olvidar la clase collapse (además de navbar-collapse o accordion-collapse): el contenido queda siempre visible y nunca se oculta.
- No incluir navbar-expand-* : la navbar se colapsa incluso en pantallas grandes.
- Repetir el mismo id en varios paneles del accordion: solo el primero responde correctamente.

- Olvidar cargar el bundle JavaScript de Bootstrap: los botones se ven pero no reaccionan a los clics.
- Confundir data-bs-toggle="collapse" con data-bs-toggle="dropdown" al copiar y pegar código entre componentes distintos.

11. Mini evaluación

1. ¿Qué clase determina a partir de qué ancho de pantalla la navbar deja de colapsarse?
2. ¿Qué relación existe entre data-bs-target de un botón y el id del elemento que controla?
3. Explica con tus palabras la diferencia entre lo que hace Bootstrap CSS y lo que hace Bootstrap JavaScript en la navbar.
4. ¿Qué atributo permite que, en un accordion, solo un panel esté abierto a la vez?
5. Un estudiante hizo clic en el botón hamburguesa y no pasó nada. Menciona dos causas posibles.
6. ¿Qué clase agregarías a un <li> para convertirlo en un menú desplegable dentro de la navbar?

12. Evidencia de aprendizaje

- Archivo index.html con la navbar responsive, el accordion de tres proyectos y el dropdown del reto guiado.
- Captura de pantalla del menú hamburguesa abierto en vista móvil (herramientas de desarrollador).

13. Cierre

Lo que aprendimos

- Construcción de una navbar responsive con menú colapsable.
- Uso de accordion para organizar contenido detallado.
- Diferencia entre el rol del CSS y el JavaScript de Bootstrap en componentes interactivos.

Lo que utilizaremos en la siguiente sesión

La navegación construida hoy enlaza a secciones (Proyectos, Equipo, Contacto) que aún no existen completamente. En la Sesión 4 se construirán esas secciones faltantes, incluyendo un formulario de contacto, y se integrará todo en una página final completa y coherente.

Nota para el profesor

- Confusión frecuente: alumnos que confunden el # del href de un enlace ancla (#proyectos) con el data-bs-target de un botón colapsable (#menuPrincipal); son mecanismos distintos aunque se ven parecidos.
- Observar mientras trabajan: quiénes repiten ids entre paneles del accordion.
- Errores que indican falta de comprensión: modificar data-bs-toggle esperando que cambie el color del botón (confunden comportamiento con apariencia).
- Preguntas de verificación: '¿Qué pasaría si dos botones distintos apuntan al mismo id?'
- Contenido omitible si el grupo avanza lento: el dropdown del reto guiado puede quedar como tarea para la próxima clase.
- Actividad adicional si terminan antes: agregar una cuarta sección al accordion con información general del laboratorio (historia, misión).

Sesión 4 — Integración y proyecto final

1. Título

Cerrando el sitio: equipo, contacto y entrega final

2. Objetivo de aprendizaje

- Integrar todos los conocimientos previos (grid, componentes, navegación) en una página única y coherente.
- Construir un formulario de contacto funcional en apariencia, usando clases de formulario de Bootstrap.
- Implementar la sección de equipo del laboratorio y un footer completo.
- Resolver, con apoyo mínimo, la integración final del proyecto de manera responsive.

3. Conocimientos previos

Se retoma el archivo index.html completo de la Sesión 3: navbar responsive, alert, fila de proyectos con cards, accordion de detalle técnico y dropdown. Hoy se completan las secciones 'Equipo' y 'Contacto' que la navbar ya referencia, y se agrega un footer.

4. Conceptos nuevos

- form-control: clase base para campos de formulario (texto, email, textarea).
- form-label: etiqueta asociada visualmente y semánticamente a un campo.
- input-group: agrupa un campo con un elemento adicional (ícono, texto, botón).
- form-check: casillas de verificación y radios con estilo de Bootstrap.
- Validación visual básica: clases is-valid / is-invalid y feedback (valid-feedback / invalid-feedback).
- Footer con utilities: combinación de bg-dark, text-light, py-* y el grid ya conocido para un pie de página de varias columnas.

5. Distribución de los 80 minutos

Tiempo	Actividad	Descripción
5–10 min	Recuperación de conocimientos previos	Se revisa el sitio completo construido hasta la Sesión 3: navbar, cards, accordion.
10–15 min	Explicación de conceptos nuevos	Explicación de form-control, form-label e input-group con un ejemplo mínimo de campo de texto.
20–25 min	Demostración guiada	El profesor construye en vivo la sección de equipo (cards de integrantes) y el inicio del formulario de contacto.
25–30 min	Práctica del estudiante	Los estudiantes completan el formulario de contacto y agregan el footer final del sitio.

Tiempo	Actividad	Descripción
5–10 min	Reto de consolidación	Agregar validación visual básica al campo de correo electrónico del formulario.
5 min	Cierre y comprobación	Revisión final proyectada del sitio completo en móvil y escritorio; entrega de evidencias.

6. Preparación del entorno

1. Abrir la carpeta laboratorio-robotica con el trabajo acumulado de las tres sesiones anteriores.
2. Confirmar que los enlaces de la navbar (#proyectos, #equipo, #contacto) siguen presentes.
3. No se crean archivos nuevos; se agregan dos nuevas secciones y el footer al final de index.html.
4. Mantener Live Server activo para verificar cada bloque conforme se construye.

7. Tutorial paso a paso

Paso 1 — Construir la sección de equipo

Qué vamos a hacer

Creamos una nueva row con id="equipo" (para que el enlace de la navbar funcione) mostrando tres integrantes del laboratorio en formato de card.

Código — después del accordion de la Sesión 3

```
<div class="row mt-5" id="equipo">
  <div class="col-12">
    <h2>Equipo</h2>
  </div>
  <div class="col-sm-6 col-lg-4 mt-3">
    <div class="card shadow-sm h-100 text-center">
      <div class="card-body">
        <h5 class="card-title">Coordinación general</h5>
        <p class="card-text">Responsable del diseño y planeación de proyectos.</p>
      </div>
    </div>
  </div>
</div>
```

Dónde colocarlo

Justo después del cierre del del accordion (Sesión 3, Paso 5), dentro del mismo container principal.

Qué deberías observar

Al hacer clic en 'Equipo' en la navbar, el navegador se desplaza directamente hasta esta sección gracias al id.

¿Por qué funciona?

El id="equipo" en la row coincide con el href="#equipo" del enlace de navegación creado en la Sesión 3; el navegador realiza scroll automático hacia ese elemento.

Paso 2 — Completar el equipo con las columnas restantes

Qué vamos a hacer

Replicamos el patrón de card del Paso 1 para dos integrantes adicionales, reutilizando exactamente la misma estructura.

Código — dos columnas adicionales dentro de la misma row

```
<div class="col-sm-6 col-lg-4 mt-3">
  <div class="card shadow-sm h-100 text-center">
    <div class="card-body">
      <h5 class="card-title">Desarrollo de firmware</h5>
      <p class="card-text">Programación de microcontroladores y sensores.</p>
    </div>
  </div>
</div>
<div class="col-sm-6 col-lg-4 mt-3">
  <div class="card shadow-sm h-100 text-center">
    <div class="card-body">
      <h5 class="card-title">Diseño mecánico</h5>
      <p class="card-text">Estructuras, chasis y ensamble de plataformas móviles.</p>
    </div>
  </div>
</div>
```

Qué deberías observar

Tres tarjetas de equipo alineadas, con el mismo comportamiento responsive ya conocido desde la Sesión 1.

¿Por qué funciona?

Al reutilizar exactamente col-sm-6 col-lg-4 y la estructura de card ya practicada, el estudiante aplica sin repetición explicaciones ya asimiladas en sesiones anteriores.

Paso 3 — Construir el formulario de contacto

Qué vamos a hacer

Creamos la sección de contacto con un formulario de nombre, correo y mensaje.

Código — nueva sección con id="contacto"

```
<div class="row mt-5" id="contacto">
  <div class="col-12 col-lg-8 mx-auto">
    <h2>Contacto</h2>
    <form>
      <div class="mb-3">
        <label for="nombre" class="form-label">Nombre completo</label>
        <input type="text" class="form-control" id="nombre">
      </div>
      <div class="mb-3">
        <label for="correo" class="form-label">Correo electrónico</label>
        <input type="email" class="form-control" id="correo">
      </div>
      <div class="mb-3">
        <label for="mensaje" class="form-label">Mensaje</label>
        <textarea class="form-control" id="mensaje" rows="4"></textarea>
      </div>
      <button type="submit" class="btn btn-primary">Enviar</button>
    </form>
  </div>
</div>
```

Qué deberías observar

Un formulario centrado, con campos de ancho consistente y un botón morado 'Enviar', ya alineado a la rejilla del sitio.

¿Por qué funciona?

mx-auto centra el bloque de formulario dentro de la fila; form-control estandariza el alto, el borde y el padding de todos los campos, sin necesidad de CSS adicional.

Paso 4 — Agregar validación visual básica al correo

Qué vamos a hacer

Añadimos clases de validación visual al campo de correo para mostrar cómo Bootstrap comunica errores sin JavaScript adicional.

ANTES

```
<input type="email" class="form-control" id="correo">
```

DESPUÉS

```
<input type="email" class="form-control is-invalid" id="correo">
<div class="invalid-feedback">
  Por favor ingresa un correo electrónico válido.
</div>
```

Qué deberías observar

El campo se resalta con borde rojo y aparece un mensaje de error debajo, incluso sin haber enviado el formulario (por ser una demostración estática).

¿Por qué funciona?

is-invalid es una clase puramente visual; en un proyecto real se agregaría/quitaría dinámicamente mediante JavaScript o validación del navegador (required, pattern), pero a nivel de este curso basta con comprender su efecto visual.

Paso 5 — Construir el footer final

Qué vamos a hacer

Cerramos el sitio con un pie de página de tres columnas, fuera del container principal para que ocupe todo el ancho de la pantalla.

Código — después del cierre del container principal, antes de </body>

```
<footer class="bg-dark text-light py-4 mt-5">
  <div class="container">
    <div class="row">
      <div class="col-12 col-md-4">
        <h5>Laboratorio de Robótica</h5>
        <p>Universidad de Montemorelos</p>
      </div>
      <div class="col-12 col-md-4">
        <h5>Enlaces</h5>
        <p><a href="#proyectos" class="text-light">Proyectos</a></p>
        <p><a href="#equipo" class="text-light">Equipo</a></p>
      </div>
      <div class="col-12 col-md-4">
        <h5>Contacto</h5>
```

```
        <p>laboratorio.robotica@um.edu.mx</p>
      </div>
    </div>
  </div>
</footer>
```

Qué deberías observar

Una franja oscura de ancho completo al final de la página, con tres columnas de información que se apilan en móvil, siguiendo exactamente el mismo comportamiento de grid aprendido en la Sesión 1.

¿Por qué funciona?

Al colocar footer fuera del container principal pero con su propio container interno, se logra que el fondo oscuro ocupe todo el ancho de la ventana mientras el contenido permanece centrado y alineado al resto del sitio.

8. Experimento rápido

Cambia is-invalid por is-valid en el campo de correo y ajusta el texto de invalid-feedback a valid-feedback.

- ¿Qué color adopta el campo ahora?
- ¿Qué diferencia notas entre el mensaje de error y el mensaje de confirmación en cuanto a color y tono del texto?

9. Reto guiado

Problema

Integrar completamente el sitio: revisar que la navbar, el alert, los proyectos, el accordion, el equipo, el formulario de contacto y el footer convivan sin romper el layout responsive en ningún punto de quiebre.

Requisitos

- Los tres enlaces de la navbar deben desplazar correctamente a sus secciones (#proyectos, #equipo, #contacto).
- Ninguna sección debe desbordar horizontalmente en móvil (sin scroll lateral).
- El formulario debe verse centrado y legible en escritorio y en móvil.

Pistas

- Usa las herramientas de desarrollador para revisar cada breakpoint uno por uno, de más pequeño a más grande.
- Si algo se desborda, revisa si olvidaste una row o un container en esa sección.

Solución propuesta

No existe un único bloque de código para este reto: la solución consiste en recorrer sistemáticamente el archivo index.html completo, verificando que cada sección siga el patrón container → row → col-* aprendido desde la Sesión 1, y corrigiendo cualquier row huérfana o columna sin row contenedora.

10. Errores frecuentes

- Cerrar el container principal antes del footer y luego intentar meter el footer dentro de él por error de indentación, rompiendo el ancho completo del pie de página.
- Olvidar el atributo for en <label> apuntando al id correcto del input, lo que rompe la asociación de accesibilidad entre etiqueta y campo.

- Usar is-invalid en todos los campos por accidente, haciendo parecer que todo el formulario tiene errores.
- No agregar mx-auto al centrar el formulario, dejándolo pegado al lado izquierdo en pantallas grandes.
- Duplicar el id="contacto" o id="equipo" en más de un elemento, lo que hace que el navegador salte siempre al primero que encuentra.
- Dejar el formulario sin ningún type="submit" en el botón, lo que puede confundir su propósito visual aunque no afecte el estilo.

11. Mini evaluación

1. ¿Qué relación debe existir entre el href de un enlace de navbar y el id de la sección a la que apunta?
2. ¿Qué clase estandariza la apariencia de un campo de texto, correo o textarea en Bootstrap?
3. ¿Qué combinación de utilities usarías para centrar horizontalmente un bloque de ancho limitado (por ejemplo col-lg-8) dentro de su fila?
4. ¿Qué clase mostrarías junto a is-invalid para explicar al usuario qué está mal en un campo?
5. Un estudiante coloca el footer dentro del container principal por error. ¿Qué problema visual es más probable que aparezca?
6. Enumera, en orden, las secciones que conforman la página final del laboratorio de robótica construida a lo largo de las 4 sesiones.
7. ¿Por qué es importante revisar el sitio completo en varios breakpoints antes de considerar terminado el proyecto?

12. Evidencia de aprendizaje

- Archivo index.html final, con todas las secciones integradas: navbar, alert, proyectos, accordion, equipo, formulario de contacto y footer.
- Capturas de pantalla del sitio completo en tres anchos (móvil, tablet, escritorio).
- Breve reflexión escrita (2-3 líneas) sobre qué parte del proyecto resultó más difícil de integrar y por qué.

13. Cierre

Lo que aprendimos

- Construcción de formularios estilizados con Bootstrap, incluyendo validación visual básica.
- Integración completa de layout, componentes y navegación en un solo sitio coherente.
- Verificación sistemática del comportamiento responsive en un proyecto completo.

Lo que utilizaremos en la siguiente sesión

Con la Unidad 1 concluida, el sitio del laboratorio de robótica queda como base reutilizable: en unidades posteriores del curso podrá ampliarse con nuevas tecnologías de frontend, conectarse a un backend o versionarse con control de versiones, según la planeación general de la materia.

Nota para el profesor

- Confusión frecuente: alumnos que no entienden por qué el footer 'se ve angosto' cuando quedó accidentalmente dentro del container principal.
- Observar mientras trabajan: quiénes prueban el sitio solo en el tamaño de ventana por defecto, sin reducir el ancho para verificar responsividad.

- Errores que indican falta de comprensión: usar is-invalid sin haber comprendido que es una clase puramente visual, esperando que valide datos reales.
- Preguntas de verificación: '¿Qué pasa si cambio el id de la sección de equipo pero no actualizo el enlace de la navbar?'
- Contenido omitible si el grupo avanza lento: la validación visual (Paso 4) puede quedar como ejercicio opcional o tarea.
- Actividad adicional si terminan antes: pedir que agreguen un cuarto campo al formulario (por ejemplo, un select de 'Área de interés') usando form-select.